

TrueSight Frontend Documentation

This document provides a detailed explanation of the TrueSight Streamlit frontend application, its file structure, core pages, and component functions.

1. Overview

The TrueSight frontend is a **Streamlit** web application designed to act as a user-friendly interface for the TrueSight backend API. Its primary purpose is to:

1. Allow users to upload videos for deepfake analysis.
2. Handle the asynchronous polling required to get results.
3. Display the final "Real" or "Fake" verdict in a clear, thematic way.
4. Provide a dashboard for system-wide statistics and job history.
5. Conditionally display (placeholder) explainability visualizations for "Fake" videos.

The application is built around a **multi-page app (MPA)** architecture, which uses Streamlit's `st.query_params` to manage navigation between pages, all driven from the main `src/app/app.py` file.

Core Aesthetic: The UI is built on a **"cyber-forensic"** dark theme. The entire theme is **dynamic** and switches between two modes based on the analysis result:

- **Red/Black:** The default state, and the theme for "Fake" verdicts.
- **Green/Black:** The theme for "Real" verdicts.

2. Core Application (`src/app/app.py`)

This is the main entry point for the application (`streamlit run src/app/app.py`). It is responsible for global setup, CSS, navigation, and routing.

Global Variables

- `BACKEND_URL` : The API root (e.g., `http://localhost:5000`), loaded from an environment variable.
- `COLOR_*` : A set of constants (`COLOR_RED` , `COLOR_GREEN` , etc.) that define the application's color palette.

Session State

Streamlit's `st.session_state` is used to maintain user context across page reloads:

- `st.session_state.current_job_id` : Stores the ID of the most recently submitted job.
- `st.session_state.job_result` : Stores the complete JSON response from `GET /results/{job_id}` .
- `st.session_state.job_status` : Tracks the current state of the job (`queued` , `processing` , `completed` , `failed`).

- `st.session_state.verdict` : Stores the final verdict (`Real` or `Fake`). **This is critical**, as it controls the dynamic Red/Green theme switch.

Key Functions

`inject_global_css()` This function is responsible for the entire visual aesthetic.

- It dynamically sets the `verdict_color` based on `st.session_state.verdict` .
- It injects a large block of CSS using `st.markdown(unsafe_allow_html=True)` .
- This CSS imports the **Orbitron** and **Roboto** fonts from Google Fonts.
- It defines all custom classes, such as `.verdict-readout` (for the 64px verdict text), `.sidebar-link` (for the navigation), and the pulsing animations for progress bars (`@keyframes pulse-progress`).

`display_sidebar_nav(current_page_name)` Renders the left-hand sidebar navigation menu.

- It displays the "TrueSight" title.
- It iterates over the `PAGES` dictionary and creates a `st.button` for each page.

`Maps_to(page_name)` This is the core navigation function.

- When a sidebar button is clicked, this function is called.
- It updates the URL's query parameter (e.g., `?page=Upload Video`).
- It calls `st.rerun()` , which forces Streamlit to reload the script.

`render_home_page()` Renders the main landing page (the default view).

- Displays the large "TrueSight" hero title and subtitle.
- Shows a "Start New Analysis" button that calls `Maps_to("Upload Video")` .
- Displays static `st.metric` boxes for "Model Accuracy" and "Critical Threshold".

Main Execution (`if __name__ == '__main__':`): This is the **Application Router**.

1. Calls `st.set_page_config` to set the browser tab title (`page_title=...`).
2. Calls `inject_global_css()` to apply the theme.
3. Reads the `page` from the URL (`st.query_params`).
4. Calls `display_sidebar_nav()` to show the navigation.
5. Uses an `if/elif/else` block to determine which page's `render_` function to import and execute.

3. Pages (`src/app/pages/`)

`pages/upload.py`

This page manages the entire video submission and polling process.

- `render_upload(backend_url)` :
 - Renders the `st.file_uploader` widget.
 - If a file is uploaded, it validates the 500 MB size limit.

- When the "Initiate Analysis" button is clicked, it makes a `POST` request to `/upload` with the file.
- On success (HTTP 202), it saves the new `job_id` to `st.session_state` and calls `st.rerun()`.
- `poll_status_ui(backend_url)` :
 - This function is called if the app detects a job is `queued` or `processing`.
 - It runs a `while` loop that calls `GET /status/{job_id}` every 2 seconds.
 - It updates the UI with the current status (e.g., "PROCESSING") and a pulsing progress bar.
 - If the status changes to `completed` or `failed`, it breaks the loop and calls `get_results()`.
- `get_results(...)` :
 - Called after polling is complete.
 - Makes a `GET` request to `/results/{job_id}` to fetch the final JSON data.
 - Saves the data to `st.session_state.job_result` and the verdict to `st.session_state.verdict`.
 - Calls `Maps_to("Results / Dashboard")` to show the user the final report.

`pages/results.py`

This page displays the final verdict for the *current* job and a history dashboard for *all* jobs.

- `render_results(backend_url)` :
 - **Part 1: Analysis Report:** If `st.session_state.job_result` is available, it renders the main report.
 - Uses `st.columns` to create the large **Verdict Card** (e.g., "FAKE" and "73.45%").
 - Displays the critical **"Threshold 0.3 Applied"** message.
 - Calls `render_processing_metrics()` to show the timing/size info.
 - Shows a "View Forensic Visualization" button to navigate to the Explainability page.
 - **Part 2: System Dashboard:** This part *always* runs.
 - Calls `GET /stats` to render the `st.metric` boxes (Total Jobs, Completed Jobs, etc.).
 - Calls `GET /jobs` to get the history table.
 - Uses `st.selectbox` to provide filters for "limit" and "status".
 - Displays the job history in a `st.dataframe`.

`pages/explainability.py`

This page shows the Grad-CAM visualizations. It is **highly conditional**.

- `render_explainability()` :
 - Checks `st.session_state.verdict`. If the verdict is `"Real"`, it displays a simple, Green-themed "Authenticity Verified" message and stops.
 - If the verdict is `"Fake"`, it switches to the Red-themed view.
 - It renders `st.slider` widgets for "Heatmap Opacity" and "Frame Scrubber".

- It calls `get_fake_grad_cam_data()` to generate placeholder images.
- It calls `render_video_player()` to display the placeholder heatmap visualization.
- `get_fake_grad_cam_data()` :
 - A **placeholder function** that generates 8 random `numpy` arrays. This simulates the heatmap data that the backend will eventually provide.

`pages/user_guide.py`

This is a simple, static page.

- `render_user_guide()` :
 - Reads the `docs/USER_GUIDE.md` file from the disk.
 - Renders the content using `st.markdown` .
 - Includes a hardcoded "Quick Start" section as a fallback.

4. Reusable Components (`src/app/components/`)

`components/metrics_display.py`

- `render_processing_metrics(result, verdict_color)` :
 - A simple component that takes the `result` object and a `verdict_color` as input.
 - It displays the "Processing Time", "File Size", and "Raw Probability" in three `st.columns` .

`components/video_player.py`

- `render_video_player(...)` :
 - A **placeholder component** that simulates the visual output.
 - It takes the `current_frame_index` , `grad_cam_frames` (placeholders), and `opacity` as input.
 - It uses `cv2` and `numpy` to create a blank frame and a fake heatmap (from `cv2.COLORMAP_JET`).

`result = result if result is not None else {}`